



**UTM**  
UNIVERSITI TEKNOLOGI MALAYSIA

**FACULTY OF COMPUTING**  
UTM Johor Bahru

Tutorial 4

Special Topic in Data Engineering (SECP3843)

| Members              | Matric No |
|----------------------|-----------|
| BRENDAN CHIA YAN FEI | A23CS0211 |
| CHEW CHIU XIAN       | A23CS0061 |
| NG YU HIN            | A23CS0148 |

Lecturer: Dr. Aryati Binti Bakri

# 1. INTRODUCTION

---

## 1.1 What is AI-Assisted Data Engineering

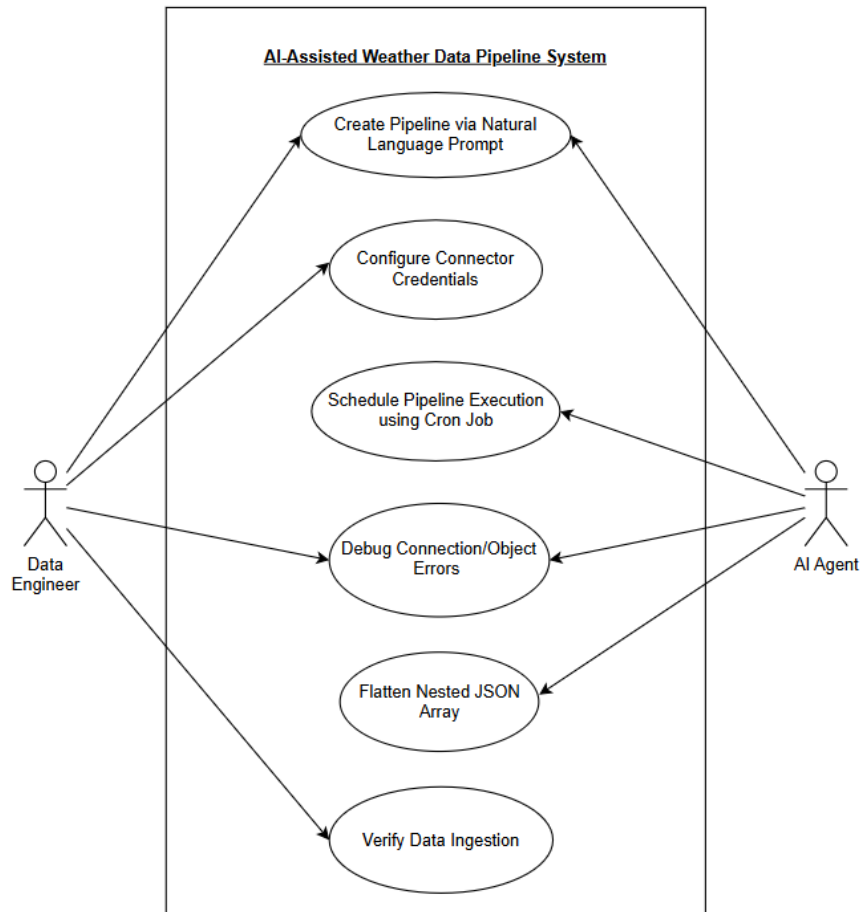
AI-assisted data engineering is an approach where natural language prompts and AI agents are used to design, build, and manage data pipelines instead of manually writing pipeline configuration or ETL code line-by-line. The AI agent interprets the user's intent regarding the source, destination, schedule, and transformations. It then generates the underlying pipeline configuration and helps debug errors when they occur, which significantly lowers the technical barrier for connecting APIs to data warehouses.

## 1.2 What AI Agent Used in the Tutorial

In this assignment, the AI agent used was `express.dev`'s built-in AI assistant (Express), which functions similarly to Claude in that it accepts natural language instructions and translates them into a working pipeline on a visual flow canvas. The agent was used to: create the initial pipeline, diagnose a Snowflake authentication/object error, identify a data-structure (nested JSON) issue causing null values, and generate a corrected transform to flatten the API response into proper relational columns.

# 2. USE CASE DIAGRAM

---



*Figure 1: Use Case Diagram for the AI-Assisted Weather Data Pipeline System*

As shown in **Figure 1**, the system encompasses six distinct use cases. In the first use case, Create Pipeline via Natural Language Prompt, both the Data Engineer and the AI Agent collaborate: the engineer submits a natural language statement specifying the target warehouse and execution frequency, while the AI Agent parses the request to initialise the source, transformation, and sink configurations on the flow canvas. The second use case, Configure Connector Credentials, is performed solely by the Data Engineer, who manually inputs Snowflake connection parameters including the virtual warehouse name, database, schema, and role to authenticate database interactions securely.

The third use case, Schedule Pipeline Execution using Cron Job, is handled entirely by the AI Agent, which establishes and manages an automated cron scheduling block configured to trigger the pipeline every two minutes. The fourth use case, Debug Connection/Object Errors, is a joint activity where the Data Engineer and AI Agent run diagnostic SQL commands to validate that all required warehouse objects, schemas, and account privileges exist. The fifth use case, Flatten Nested JSON Array, is performed by the AI Agent, which constructs an in-

memory transformation map to expand nested JSON payload values into clean relational rows and columns. Finally, the sixth use case, Verify Data Ingestion, is conducted by the Data Engineer, who runs data preview queries in Snowflake to confirm that the ingested columns match the target schema and that the pipeline executed successfully.

### 3. IMPLEMENTATION STEPS

---

#### 3.1 Pipeline Creation Prompt

The pipeline was initiated using a natural language prompt submitted to the Express AI agent on the `express.dev` platform. The prompt stated: *"I want to create a pipeline from the weather API at data.gov.my to my Snowflake table WEATHER\_DATA. It should run every 2 minutes."* As illustrated in **Figure 2**, the AI agent immediately interpreted this instruction and began gathering information about both connectors, the Malaysia Weather API data source and the Snowflake destination. The agent autonomously created a source node connecting to the Malaysia Weather API (`data.gov.my`), configured a Snowflake sink, and established a cron scheduling block to trigger the pipeline every two minutes. This demonstrated the core capability of AI-assisted data engineering, wherein a single natural language instruction replaces the manual configuration of multiple pipeline components.

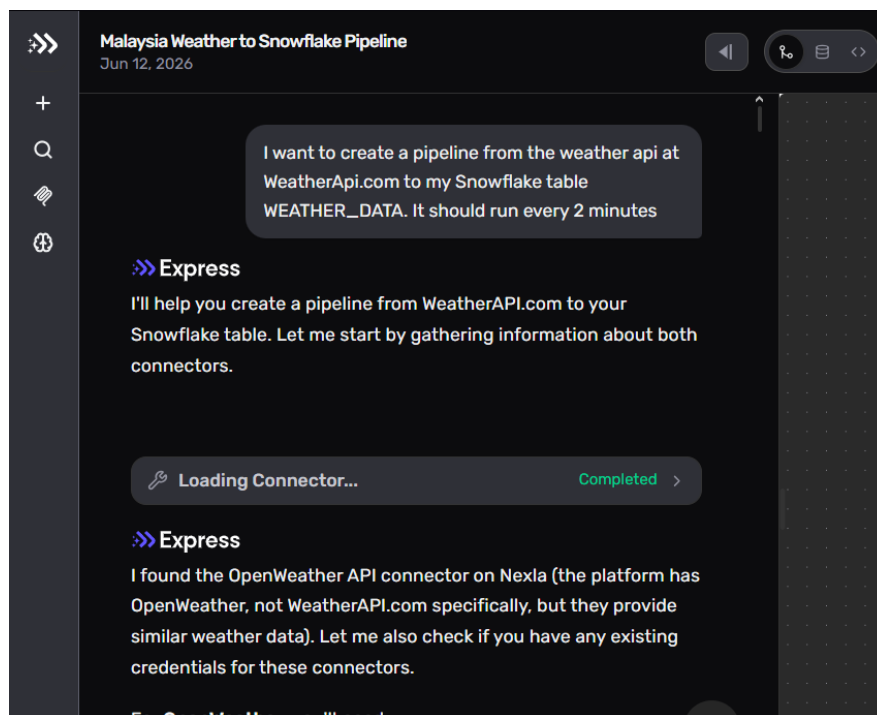


Figure 2: Prompting the AI agent in `express.dev`

### 3.2 Snowflake Credential Setup

A Snowflake credential was created using the following configuration:

- Warehouse: WEATHER\_MALAYSIA
- Database: weather\_malaysia
- Schema: weather\_data
- Role: ACCOUNTADMIN

### 3.3 Debugging Connection Errors

Initial pipeline validation failed with the error message *"Could not validate credential for Snowflake."* Working interactively with the AI agent, the team performed a systematic series of diagnostic SQL commands to isolate the root cause. The following checks were carried out: **SHOW WAREHOUSES** confirmed that the WEATHER\_MALAYSIA warehouse existed; **SHOW DATABASES** confirmed that the weather\_malaysia database was present; **SHOW SCHEMAS IN DATABASE weather\_malaysia** confirmed that the weather\_data schema had been created; and **SELECT CURRENT\_ROLE()** confirmed that the active role was ACCOUNTADMIN, thereby ruling out privilege-related issues as the cause of failure. Through this AI-guided diagnostic process, the connection error was eventually resolved by correcting the account identifier format used in the credential configuration.

### 3.4 Flattening Nested JSON Data

After the Snowflake connection was established, an additional data quality issue was identified: the raw API response returned a nested JSON array structure, which caused all target table columns to populate with null values instead of the expected weather data fields. The AI agent diagnosed this structural mismatch and generated an in-memory transformation map to expand the nested JSON payload into distinct, flat relational columns compatible with the WEATHER\_DATA table schema. This transformation step was critical to ensuring that the ingested data was properly structured for downstream querying and analysis. The complete pipeline flow resulting from these steps is visualised in Figure 3, which shows the full canvas of nodes from the Malaysia Weather API source through the flattening transform to the final Snowflake sink.

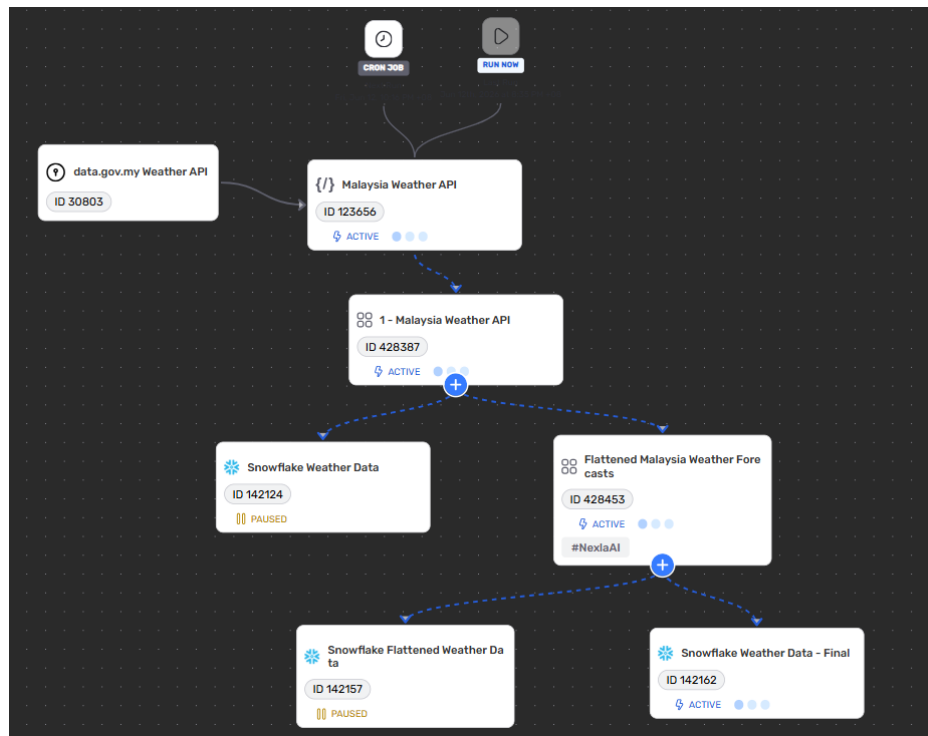


Figure 3: AI-Generated ETL Data Pipeline Flow Canvas in Express.dev.

## 4. RESULTS AND EXPLANATION

### 4.1 Pipeline Execution and Warehouse Status

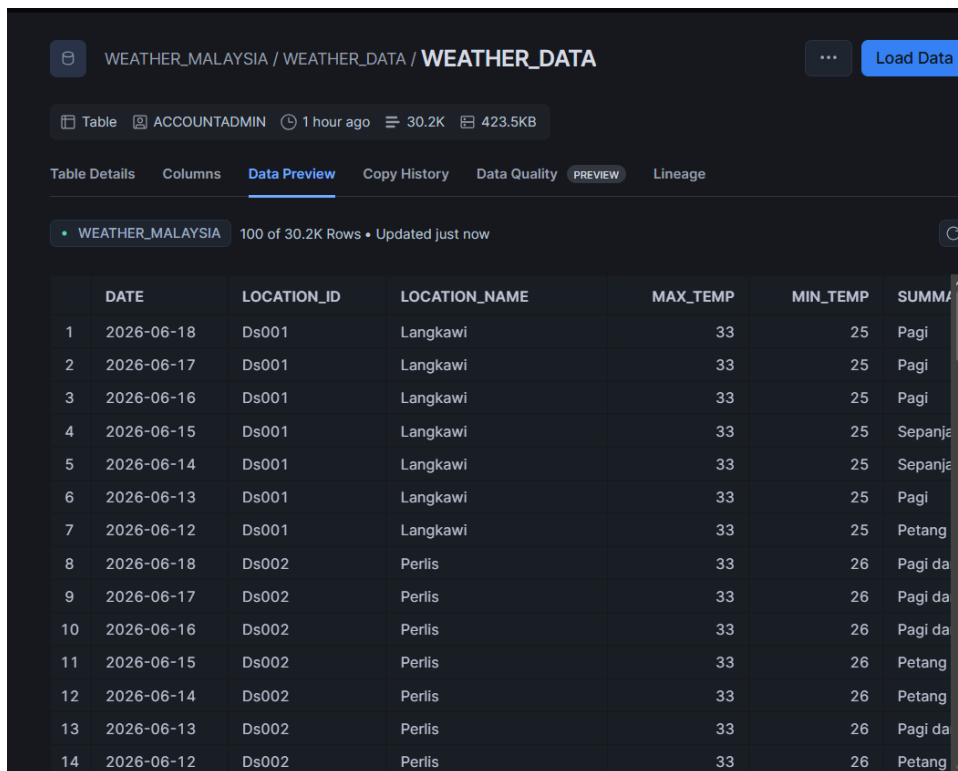
Upon successfully resolving all connection and data structure issues, the pipeline was activated and the Snowflake compute engine was confirmed to be running. As shown in **Figure 5**, the Snowflake Warehouses dashboard displays the WEATHER\_MALAYSIA warehouse with a Started status and 1 cluster running, indicating that the compute engine was active and ready to receive data.

| NAME                       | TYPE     | SIZE    | STATUS    | CLUSTERS | RUNNING | QUEUED | TABLES | OWNER |
|----------------------------|----------|---------|-----------|----------|---------|--------|--------|-------|
| WEATHER_MALAYSIA           | Standard | X-Small | Started   | 1/1      | 1       | 0      | —      | ACC   |
| TLC_WH                     | Standard | X-Small | Suspended | 0/1      | 0       | 0      | —      | ACC   |
| SYSTEM\$STREAMLIT_NOTEB... | Standard | X-Small | Suspended | 0/10     | 0       | 0      | —      | ACC   |

Figure 4: WEATHER\_MALAYSIA compute engine is started and running.

## 4.2 Data Ingestion Verification

To verify the end-to-end success of the pipeline, a data preview was conducted within the Snowflake interface on the WEATHER\_DATA table. As illustrated in **Figure 6**, the table successfully contains 30,200 rows (423.5 KB) of flattened weather forecast records ingested from the Malaysia Weather API. The data preview confirms that all expected columns are correctly populated, including DATE, LOCATION\_ID, LOCATION\_NAME, MAX\_TEMP, MIN\_TEMP, SUMMARY\_WHEN, SUMMARY\_FORECAST, MORNING\_FORECAST, AFTERNOON\_FORECAST, NIGHT\_FORECAST column.



|    | DATE       | LOCATION_ID | LOCATION_NAME | MAX_TEMP | MIN_TEMP | SUMMARY_FORECAST |
|----|------------|-------------|---------------|----------|----------|------------------|
| 1  | 2026-06-18 | Ds001       | Langkawi      | 33       | 25       | Pagi             |
| 2  | 2026-06-17 | Ds001       | Langkawi      | 33       | 25       | Pagi             |
| 3  | 2026-06-16 | Ds001       | Langkawi      | 33       | 25       | Pagi             |
| 4  | 2026-06-15 | Ds001       | Langkawi      | 33       | 25       | Sepanjan         |
| 5  | 2026-06-14 | Ds001       | Langkawi      | 33       | 25       | Sepanjan         |
| 6  | 2026-06-13 | Ds001       | Langkawi      | 33       | 25       | Pagi             |
| 7  | 2026-06-12 | Ds001       | Langkawi      | 33       | 25       | Petang           |
| 8  | 2026-06-18 | Ds002       | Perlis        | 33       | 26       | Pagi dan         |
| 9  | 2026-06-17 | Ds002       | Perlis        | 33       | 26       | Pagi dan         |
| 10 | 2026-06-16 | Ds002       | Perlis        | 33       | 26       | Pagi dan         |
| 11 | 2026-06-15 | Ds002       | Perlis        | 33       | 26       | Petang           |
| 12 | 2026-06-14 | Ds002       | Perlis        | 33       | 26       | Petang           |
| 13 | 2026-06-13 | Ds002       | Perlis        | 33       | 26       | Pagi dan         |
| 14 | 2026-06-12 | Ds002       | Perlis        | 33       | 26       | Petang           |

*Figure 5: Data preview in Snowflake confirming successful ingestion of Malaysia weather data*

## 5. REFLECTION

---

### 5.1 Brendan Chia Yan Fei

This tutorial has provided valuable hands-on experience in building automated data engineering workflows using modern AI-assisted tools. Working within the `express.dev` platform demonstrated how easily operational intent can be translated from simple natural language prompts directly into functional, structural pipeline architectures. A major learning takeaway for me was realizing how much AI agents can abstract away the tedious, line-by-line coding typically required to configure API connections, cron scheduling, and data warehouse synchronization, allowing me to focus on high-level system logic instead of getting bogged down in boilerplate ETL configurations. Furthermore, handling real-world bottlenecks, such as diagnosing Snowflake credential validation issues and resolving nested JSON structural anomalies, offered immense practical value. Watching the AI agent identify why data fields were returning null values and subsequently generating an in-memory transformation map to flatten the JSON payload into clear relational columns, showed me the true potential of AI-driven development.

### 5.2 Chew Chiu Xian

The completion of tutorial has provided me a valuable experience to construct an automated data workflows using AI-assisted tools such as Express AI. Utilizing the `express.dev` platform shows how operational intent can be translated directly into structural pipeline architecture, such as automated time scheduling, API source ingestion, and target warehouse connections without the need to code line-by-line. A major part of the project involved fixing a problem where the incoming data was trapped in a messy, layered format, causing null values in the database. The AI helped find this issue and wrote a fix to unpack the data so it could fit neatly into standard database rows and columns. In conclusion, this implementation in the tutorial clearly shows that the efficiency of AI-driven development in abstracting backend configurations.

### 5.3 Ng Yu Hin

After completing this tutorial, it has provided insightful hands-on experience in building automated data pipelines using AI-assisted tools. Using the `express.dev` AI agent, a working



pipeline was successfully configured to extract data from the Malaysia Weather API and load it into a Snowflake data warehouse. The activity demonstrated how natural language prompts can be utilized to handle pipeline creation, scheduling and data transformations, such as flattening nested JSON structures. Furthermore, debugging connection issues and schema errors during the implementation offered practical lessons in troubleshooting database systems. Ultimately, this tutorial highlighted how modern AI tools can significantly simplify complex data engineering tasks.